

Bella Italia Chatbot - Midterm Report

1. Chatbot Application

Use Case and Domain

The Bella Italia Chatbot is a food ordering assistant designed for the Bella Italia restaurant.

Note: Bella Italia is a fictitious restaurant created specifically for this coursework task.

The chatbot operates in the **restaurant and food service domain**, specifically focusing on facilitating customer interactions for menu inquiries, order placement, and restaurant information retrieval.

Primary Use Case: The chatbot enables customers to interact with the restaurant through natural language conversations to:

- Browse the menu and view available items with prices and descriptions
- Place food orders using SKU (Stock Keeping Unit) codes
- View their shopping cart and review order details
- Confirm and finalize orders
- Obtain restaurant information such as operating hours and location
- Receive personalized service through name recognition

Domain of Operation: The chatbot specializes in handling restaurant-specific interactions, including:

- **Menu Management:** Displaying categorized menu items (pastas, salads, drinks) with detailed information
- **Order Processing:** Accepting orders via SKU codes, managing quantities, and maintaining a shopping cart
- **Customer Service:** Providing restaurant information and personalized greetings
- **Order Management:** Tracking cart contents, calculating totals, and processing order confirmations

The chatbot is designed to simulate a real-world restaurant ordering system where customers can place orders through a conversational interface, making it accessible and user-friendly for customers who prefer text-based interactions over traditional phone calls or in-person ordering.

2. Test Cases

Three comprehensive test cases were implemented to validate the chatbot's functionality across different user scenarios.

Each test case demonstrates specific aspects of the chatbot's behavior and validates the integration of various components.

Test Case 1: Information-Seeking Customer

Purpose: Validates that the chatbot can handle customers who only want restaurant information without placing orders.

Test Inputs:

```
hello
what are your hours?
where are you located?
exit
```

Expected Behavior:

- The chatbot recognizes the greeting intent and responds appropriately
- When asked about hours, the chatbot identifies the **restaurant_info** intent and displays the restaurant's operating hours
- When asked about location, the chatbot again recognizes **restaurant_info** intent and provides the restaurant address

- The chatbot handles multiple information queries in sequence without requiring order placement
- The conversation terminates gracefully when the user types “exit”

Intents Tested: greeting, restaurant_info

Features Validated:

- Basic greeting recognition and response generation
- Restaurant information retrieval (hours and location)
- Multiple restaurant info queries handled sequentially
- Exit functionality

Significance: This test case ensures the chatbot can serve informational queries independently, demonstrating that customers can obtain restaurant details without engaging in the ordering process.

This is crucial for customers who may be deciding whether to visit the restaurant or are simply seeking contact information.

Test Case 2: Menu Browsing Customer

Purpose: Validates menu inquiry functionality for customers who want to explore available options before ordering.

Test Inputs:

```
hi
show me menu
what can I order?
exit
```

Expected Behavior:

- The chatbot recognizes the greeting and responds with a welcome message
- When asked to “show me menu”, the chatbot identifies the `menu_inquiry` intent and displays the complete menu with categories (pastas, salads, drinks), item names, SKU codes, prices, and descriptions
- When asked “what can I order?”, the chatbot recognizes this as another `menu_inquiry` variation and displays the menu again (demonstrating response variety)
- The chatbot handles multiple menu requests without requiring order placement
- The conversation exits cleanly

Intents Tested: greeting, menu_inquiry

Features Validated:

- Menu display formatting with proper categorization
- Multiple menu requests handled correctly (testing response variety)
- Menu data extraction and formatting from JSON structure
- Pattern matching for different phrasings of menu requests (“show me menu” vs “what can I order”)

Significance: This test case verifies that menu information is correctly formatted and displayed, ensuring customers can easily browse available options.

It also demonstrates that the chatbot handles different phrasings of the same intent, making it more user-friendly and robust.

Test Case 3: Complete Ordering Customer

Purpose: Validates the complete order placement workflow from initial greeting to order confirmation.

Test Inputs:

```
hello
I'm Sarah
show me menu
order 2 SC
add 1 CS
show cart
confirm my order
exit
```

Expected Behavior:

- The chatbot greets the customer
- When the customer says “I’m Sarah”, the chatbot extracts the name using the `name_collection` intent and stores it in memory
- Subsequent responses use the customer’s name for personalization (e.g., “Nice to meet you, Sarah!”)
- The chatbot displays the menu when requested
- When the customer says “order 2 SC”, the chatbot:
 - Identifies the `place_order` intent
 - Extracts quantity (2) and SKU (SC) from the input
 - Looks up “SC” in the menu data to find “Spaghetti Carbonara”
 - Adds 2 items to the cart with the correct price (£16.50 each)
- When the customer says “add 1 CS”, the chatbot adds 1 Caesar Salad to the cart
- When asked to “show cart”, the chatbot displays:
 - All items with quantities and individual prices
 - The total price (£48.00)
- When the customer confirms the order, the chatbot:
 - Validates that the cart is not empty
 - Displays a confirmation message with the total price
 - Clears the cart
- The conversation exits gracefully

Intents Tested: `greeting`, `name_collection`, `menu_inquiry`, `place_order`, `view_cart`, `confirm_order`

Features Validated:

- Name collection and memory storage
- Personalized responses using stored name throughout the conversation
- Menu display functionality
- Order placement with quantity extraction and SKU validation
- Multiple items added to cart sequentially
- Cart viewing with formatted display showing quantities, prices, and total
- Order confirmation with cart validation and clearing
- Memory persistence across multiple interactions

Significance: This test case demonstrates the complete end-to-end ordering process, ensuring all order management features work together correctly.

It validates that the chatbot can handle complex multi-step interactions while maintaining context (customer name and cart contents) throughout the conversation.

3. Code and Data Organization

The chatbot implementation follows a well-structured, modular architecture that separates concerns and promotes maintainability.

The code organization includes several key aspects:

3.1 Data Structure Organization

JSON Configuration File (`intents.json`):

- **Intent Definitions:** All intents, patterns, and responses are stored in a centralized JSON file
- **Menu Data:** Menu items are organized by categories (pastas, salads, drinks) with structured fields (name, SKU, price, description)
- **Restaurant Information:** Operating hours and address stored in a dedicated section
- **Benefits:** This separation allows easy updates to menu items, intents, or responses without modifying code

Memory Structure (`ChatbotMemory` Class):

- **User Name:** Stored as a string attribute
- **Shopping Cart:** Implemented as a list of dictionaries, each containing `item`, `quantity`, and `price`
- **Preferences:** Optional list for future enhancements
- **Methods:** Encapsulated operations (`set_name()`, `add_to_cart()`, `get_cart_total()`, `clear_cart()`) ensure data integrity

3.2 Code Modularity

Function-Based Architecture:

- **Data Processing Functions:** `build_pattern_to_intent()`, `build_intent_to_response()`, `extract_menu_data()` transform JSON into efficient lookup structures
- **NLP Functions:** `preprocess_input()`, `match_pattern()`, `extract_quantity()` handle text processing and pattern matching
- **Response Generation:** `generate_response()` handles template selection and placeholder substitution
- **Order Management:** `find_item_by_sku()`, `format_cart_for_display()` provide cart-related utilities
- **Main Loop:** `process_user_input()` encapsulates the core processing logic, while `demo_loop()` provides the testing interface

Separation of Concerns: - **Preprocessing:** Handled independently from pattern matching

- **Pattern Matching:** Separate from response generation
- **Memory Management:** Isolated in a dedicated class
- **Order Management:** Separate functions for cart operations

3.3 Notebook Organization

The Jupyter notebook is organized into logical sections:

1. **Introduction:** Chatbot overview and implemented intents
2. **Memory Structure:** `ChatbotMemory` class implementation
3. **Core Functions:** Data structure transformation functions
4. **Text Preprocessing:** Input normalization pipeline
5. **Pattern Recognition:** Regex-based intent matching
6. **Response Generation:** Dynamic response creation
7. **Order Management:** Helper functions for cart operations
8. **Demo Loop:** Non-interactive testing interface
9. **Test Cases:** Three comprehensive test scenarios
10. **Interactive Demo:** Production-ready interface

Each section includes:

- **Markdown Documentation:** Explains purpose, implementation details, and rationale
- **Code Cells:** Implementations with docstrings
- **Test Cells:** Examples demonstrating functionality

3.4 Import Organization

All library imports are consolidated at the beginning of the notebook:

- **Standard Library:** `json`, `os`, `re`, `random`

- **External Libraries:** nltk with specific imports (WordNetLemmatizer, word_tokenize, stopwords)
- **NLTK Data Downloads:** Handled with error checking to ensure required resources are available

This organization ensures:

- **Reproducibility:** All dependencies are clearly identified
 - **Maintainability:** Easy to update or add new dependencies
 - **Clarity:** Readers can immediately see what libraries are used
-

4. Advanced Techniques

The chatbot implementation employs several advanced Natural Language Processing (NLP) and software engineering techniques:

4.1 Text Preprocessing Pipeline

Multi-Step Normalization:

1. **Punctuation Removal:** Strips punctuation while preserving apostrophes for contractions (e.g., “I’m” remains intact)
2. **Whitespace Normalization:** Collapses multiple spaces and newlines into single spaces
3. **Tokenization:** Uses NLTK’s `word_tokenize()` to split text into word tokens, handling contractions correctly

4. **Intelligent Stop Word Removal:**

- Removes common English stop words (e.g., “the”, “a”, “an”)
- **Critical Innovation:** Maintains an `important_words` set containing words essential for intent recognition that would normally be filtered (e.g., “what”, “where”, “show”, “me”, “my”, “name”, “is”, “have”, “order”, “menu”)
- This prevents pattern matching failures when crucial words are removed

5. **Advanced Lemmatization:**

- Uses WordNetLemmatizer with multiple part-of-speech attempts (verb, noun, adjective)
- **Preserves Certain Words:** Maintains a `preserve_words` set to prevent lemmatization of words that must remain in original form (e.g., “am”, “is”, “are”, “morning”, “afternoon”, “evening”)
- Handles edge cases: single-character tokens and proper nouns are preserved

Why This Matters: The preprocessing pipeline ensures that variations in user input (e.g., “Show me the menu!”, “show menu”, “showing menu”) converge to similar normalized forms that match intended patterns, significantly improving pattern matching reliability.

4.2 Priority-Based Pattern Matching

Three-Tier Priority System:

1. **Highest Priority:** `confirm_order` patterns (prevents “confirm my order” from incorrectly matching “my order” pattern)
2. **High Priority:** Specific intents (`menu_inquiry`, `name_collection`, `place_order`, `view_cart`, `restaurant_info`)
3. **Lower Priority:** General intents (`greeting`)

Pattern Types Supported:

- **Anchored Patterns** (`^pattern$`): Exact string match
- **Start-Anchored** (`^pattern`): Matches from start of string
- **End-Anchored** (`pattern$`): Matches at end of string
- **Substring Patterns:** Matches anywhere in the string

Why This Matters: This priority system prevents ambiguous matches.

For example, “hey show me menu” correctly matches `menu_inquiry` (specific) rather than `greeting` (general), ensuring more accurate intent recognition.

4.3 Dynamic Response Generation with Placeholder Substitution

Template-Based System: - Each intent has multiple response templates stored in JSON

- Random selection ensures conversation variety

- **Nine Placeholder Types:**

1. {restaurant_name}: Restaurant name
2. {name}: Customer name (with “guest” fallback)
3. {menu_list}: Formatted menu display
4. {quantity}: Order quantity
5. {item_name}: Ordered item name
6. {cart_display}: Formatted cart contents
7. {cart_total}: Total cart price
8. {restaurant_hours}: Operating hours
9. {restaurant_address}: Physical address

Substitution Order: Placeholders are replaced in a specific order to handle nested or complex placeholders correctly.

Why This Matters: This approach enables personalized, context-aware responses that incorporate user-specific information (name, cart contents) and dynamic content (menu, restaurant details) while maintaining variety through random template selection.

4.4 Capture Group Extraction

Regex Capture Groups:

- Patterns use capture groups (\w+) to extract values from user input
- **Name Extraction:** Pattern "i (\\w+)" captures names from “I’m John” or “I am Sarah”
- **Order Extraction:** Pattern "order (\\d+) ([a-zA-Z]{2})" captures both quantity and SKU from “order 2 SC”
- **Flexible Matching:** Handles patterns with 1 or 2 capture groups, defaulting quantity to 1 if not specified

Why This Matters: Capture groups enable the chatbot to extract structured information (names, quantities, SKUs) from natural language input, allowing users to provide information in a conversational manner rather than requiring rigid input formats.

4.5 Memory Persistence and Context Management

Stateful Conversation:

- The ChatbotMemory class maintains conversation state across multiple interactions
- **Name Persistence:** Once collected, the customer’s name is used in all subsequent responses
- **Cart Persistence:** Items remain in the cart across multiple interactions until explicitly cleared
- **Context Awareness:** The chatbot remembers previous interactions, enabling natural multi-turn conversations

Why This Matters: Memory persistence creates a more natural conversation experience, allowing customers to ask questions, view the menu, and place orders in any order without losing context. This mimics human conversation where context is maintained throughout the interaction.

4.6 Error Handling and Validation

Input Validation:

- **Empty Cart Check:** Prevents order confirmation when cart is empty
- **SKU Validation:** Verifies SKU codes exist in menu data before adding to cart
- **Empty Input Handling:** Provides helpful messages for empty inputs
- **Invalid Pattern Handling:** Gracefully handles regex errors and invalid patterns

Why This Matters: Robust error handling ensures the chatbot provides helpful feedback when users make mistakes or provide invalid input, improving user experience and preventing system errors.

4.7 Modular Function Design

Reusable Components:

- `process_user_input()` encapsulates the complete processing pipeline, making it reusable by both `demo_loop()` and `main_loop()`
- Helper functions (`find_item_by_sku()`, `format_cart_for_display()`) are modular and testable independently
- Clear separation between data processing, NLP, and business logic

Why This Matters: Modular design promotes code reusability, testability, and maintainability. Changes to one component don't affect others, making the codebase easier to extend and debug.

Conclusion

The Bella Italia Chatbot demonstrates a comprehensive implementation of a conversational food ordering system using advanced NLP techniques.

Through careful code organization, sophisticated text preprocessing, priority-based pattern matching, and dynamic response generation, the chatbot provides a natural, user-friendly interface for restaurant interactions. The three test cases validate the chatbot's ability to handle diverse user scenarios, from simple information queries to complete order workflows, ensuring robust functionality across different use cases.